

Amar Kanakamedala | ask20@rice.edu

Alexa Seaton | ams37@rice.edu

Devin Von Arx | dav@rice.edu

Ying Jin | yj4@rice.edu

Sahil Joshi | sj157@rice.edu

Kenneth Evans | ke1@rice.edu

Part 1: Design Principles

1. **Single Responsibility Principle:** Our MVC architecture ensures that [main.py](#) (controller) handles the startup of the application and coordinates user interactions (handled by [frontend.py](#)) and backend operations (handled by [/api](#)).
2. **Open-Closed Principle:** Software entities (modules, packages, libraries) should be open to extensions but closed for modifications.
 - a. OCRProvider's interface can be added to (extension), but we do not remove the interface functions (modification)
 - b. The same can be said for MetadataProvider.
 - c. The same can be said for DocumentRepository.
3. **Dependency Inversion Principle:** high-level modules should not depend on low-level modules/functionality but rather abstractions (interfaces).
 - a. We have an OCRServiceDep in our OCR route which depends on a function `get_OCR_service`, which returns an OCRService with an OCRProvider composed into it. This allows any OCRProvider to be injected (e.g. MistralOCRProvider for OCR Service) at runtime.
 - b. We have a MetadataServiceDep in our extract route, which depends on a function `get_metadata_service`, which returns a MetadataService with a MetadataProvider composed into it. This allows any Metadata provider to be injected (e.g. OpenAIMetadataProvider) to be injected at runtime.
 - c. We have a DocumentServiceDep in our `/api/db/process` route which depends on a function `get_document_service`, which returns a DocumentService with a DocumentRepository composed into it. This allows any DocumentRepository to be composed into it, so we don't rely on one type of database model.

Part 2: Concurrency Management

- We use an asynchronous lifespan function to ensure that we can load expensive resources before startup while also ensuring the main event loop can continue (i.e.

we ensure this operation is non-blocking IO), and we ensure that we can cleanup resources concurrently ([e.g.](#) disconnect database connections and stop running tasks).

- We flush changes when we add to metadata associated with a document to our database, which ensures persistency and consistency.
- We ensure the ocr and extract routes are async and we await async ops (e.g. reading the uploaded pdf,).

Part 3a: Design Description

When the user opens our application, they see the **Front-End**, which consists of a Search option (not yet implemented but we plan to use Elastic Search) and an Extraction option. Right now, the front-end is in `frontend/frontend.py`, which serves as both the controller and the view in our MVC architecture. The model is implemented by `src/backend`, which serves as the **Back-end**. It handles calls from the front-end when a pdf is uploaded under the extraction option, and it extracts metadata using OpenAI's GPT model and Mistral's OCR model, but these models can be replaced by any models that implement an interface which we discuss in Part 1.

Part 3b: Architecture Diagram

To be added.